NETWORK SNIFFING

Analyse de trafic réseau avec Wireshark

La Plateforme · Mai 2026

Wireshark

tshark

ARP / UDP / TCP

FTP · DNS · DHCP

SMB · TLS · HTTPS

Modèle OSI

Wireshark — Présentation & Concepts

Sniffer open-source · Trame vs Paquet · PCAP/PCAPng

WIRESHARK

› Sniffer open-source (1998)

› 3 000+ protocoles

› Interface GUI + CLI tshark

› Export PCAP / PCAPng

› Capture filtrée temps réel

```
sudo apt install wireshark
tshark
sudo wireshark
```

TRAME vs PAQUET

TRAME

› L2 Ethernet

› Adresses MAC

› LAN uniquement

PAQUET

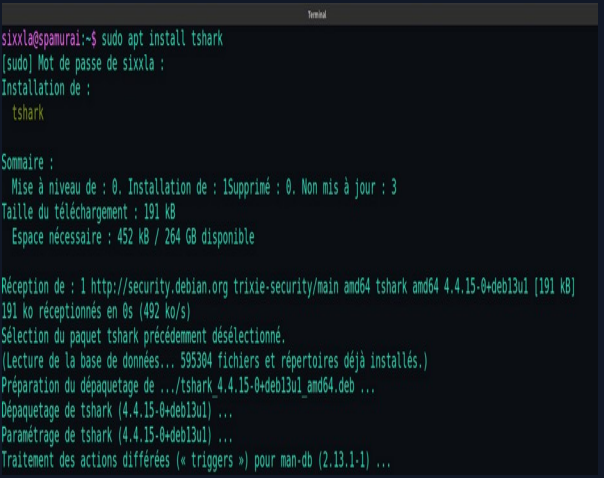
› L3 IPv4/IPv6

› Adresses IP

› Inter-réseaux

PCAP / PCAPng

.pcap — standard universel
.pcapng — multi-ifaces, commentaires, timestamps ns



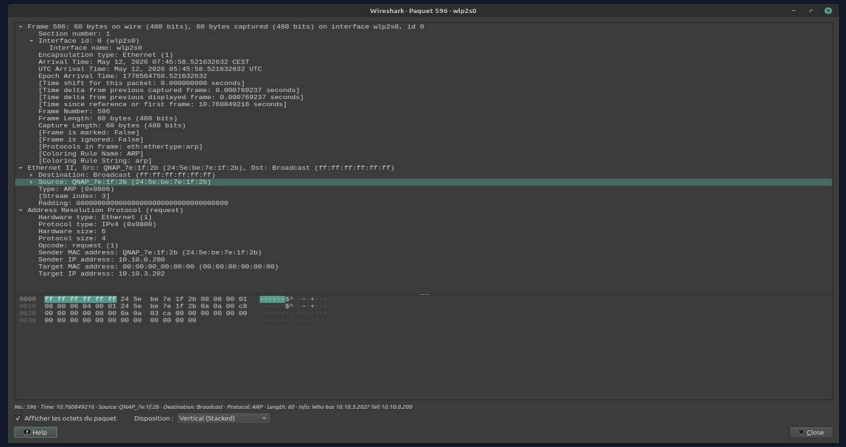
Installation tshark

Partie 1 — ARP & UDP

Address Resolution Protocol · User Datagram Protocol

ARP

Résolution IP→MAC · Type 0x0806



Capture ARP — désencapsulation Wireshark

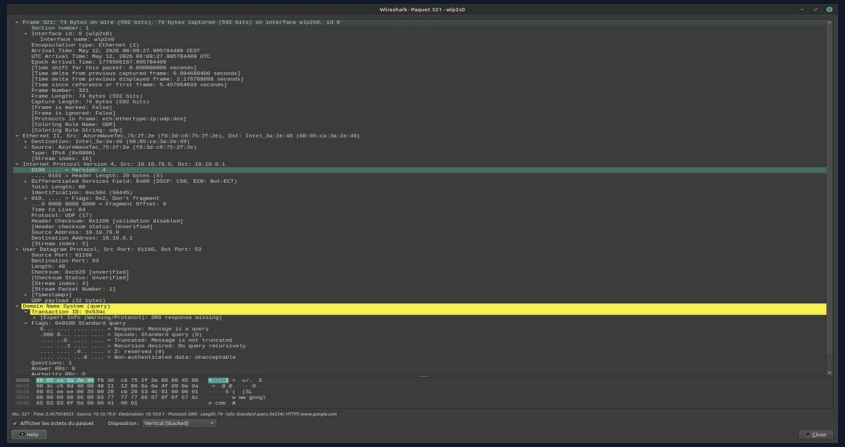
› Request : broadcast ff:ff:ff:ff:ff:ff

› Reply : unicast vers l'émetteur

› Couche 2/3 — sans ARP pas de trame Ethernet

UDP

Sans connexion · 8 octets · DNS/DHCP/NTP/mDNS



Capture UDP — paquet DNS en clair

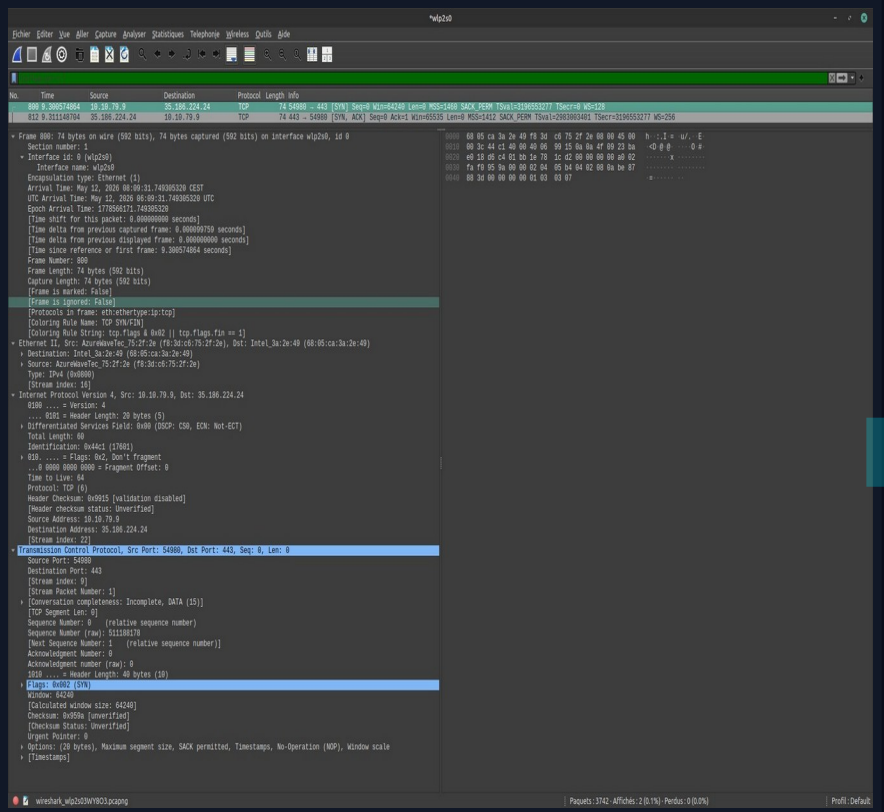
› Pas d'acquittement ni de retransmission

› En-tête 8 octets (Source/Dest port, Length, Checksum)

› Adapté DNS, DHCP, NTP, VoIP, mDNS

Partie 1 — TCP & 3-Way Handshake

Transmission Control Protocol · Connexion fiable orientée état

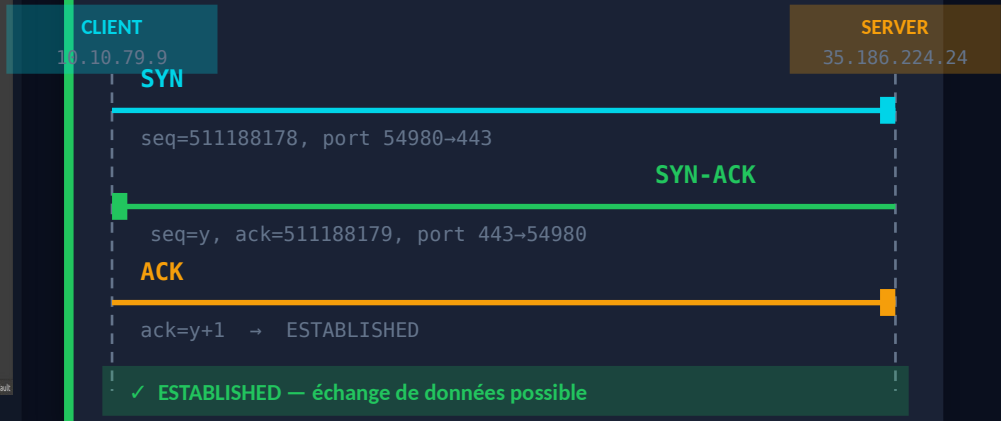


Capture TCP — SYN / SYN-ACK capturés (Wireshark)

Flags TCP

SYN	Initie la connexion
ACK	Acquittement
FIN	Fermeture
RST	Reset / erreur

3-Way Handshake



Fermeture : FIN → ACK → FIN → ACK (4 étapes, TIME_WAIT)

Filtres Wireshark

Capture filters BPF (avant) · Display filters (après) · tshark

CAPTURE (BPF)

Appliqués avant — seuls les paquets correspondants sont enregistrés

`arp`	Trames ARP uniquement
`udp`	Datagrammes UDP
`tcp port 443`	HTTPS
`udp port 53`	Requêtes DNS
`host 192.168.x.x`	IP spécifique
`not arp`	Exclure ARP

AFFICHAGE

Appliqués après — masquent sans modifier le fichier pcapng

`tcp.flags.syn==1`	Paquets SYN
`tcp.flags.fin==1`	Fermeture TCP
`dns`	Tout le DNS
`http.request.method =="GET"`	Requêtes GET
`ip.addr==x.x.x.x`	IP source/dest
`!(arp||dns||icmp)`	Exclure bruit

tshark CLI

Exemples d'options combinées

`tshark -i eth0`	Écouter eth0
`-Y "http"`	Filtre affichage
`-f "port 53"`	Filtre capture
`-c 20`	Capturer 20 paquets
`-w out.pcapng`	Sauvegarder
`-D`	Lister interfaces

Partie 2 — DHCP

Dynamic Host Configuration Protocol · Séquence DORA · UDP 67/68

Séquence DORA

D — **DISCOVER**

Client → Broadcast
Cherche un serveur DHCP

O — **OFFER**

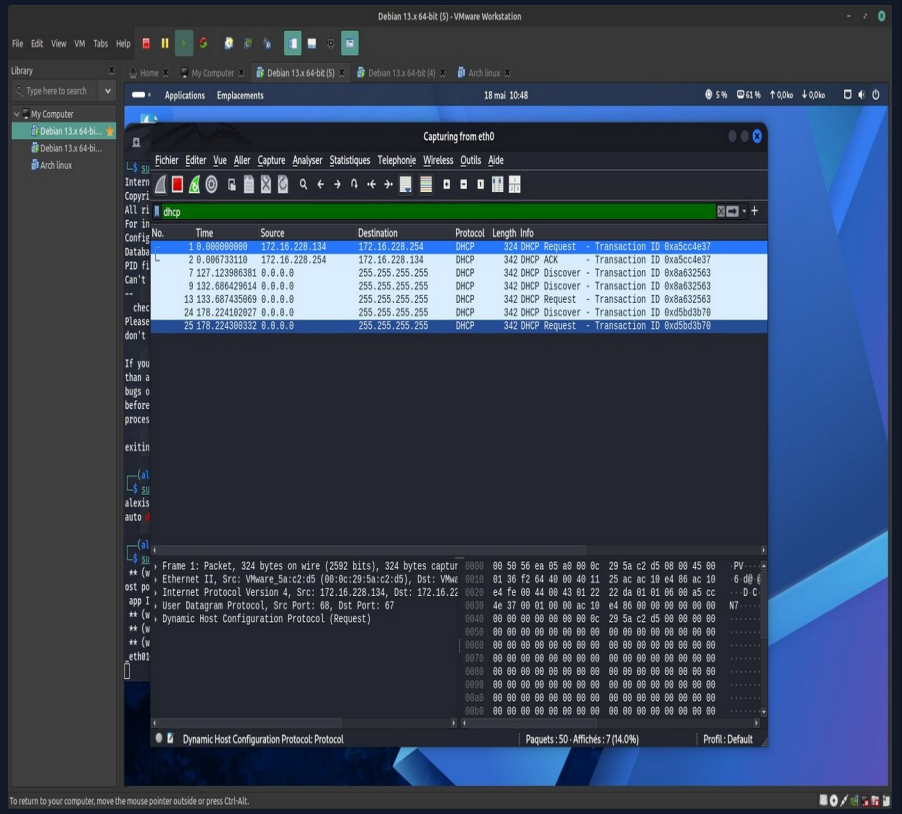
Serveur → Client
Propose une adresse IP

R — **REQUEST**

Client → Broadcast
Accepte l'offre

A — **ACK**

Serveur → Client
Confirme l'attribution



Capture DHCP — Discover · Offer · Request · ACK

Partie 2 — DNS & mDNS

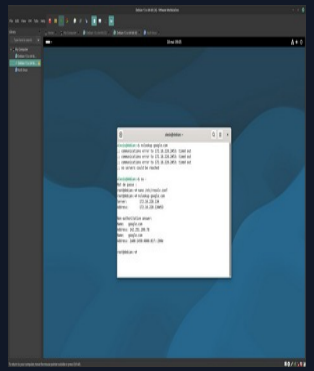
Domain Name System · Multicast DNS · UDP 53 / 5353

DNS — Port 53 UDP

› Résout noms de domaine → adresses IP

› Trafic en clair → tous les sites consultés visibles

› DoH (DNS over HTTPS) pour chiffrer



DNS client/serveur

```
sixxla@spamurai:~$ tshark -i wlp2s0 -f "port 53"
Capturing on 'wlp2s0'
    1 0.000000000   10.10.79.9 → 10.10.0.1    DNS 71 Standard query 0xe5b3 HTTPS b1.nel.goog
    2 0.000060464   10.10.79.9 → 10.10.0.1    DNS 71 Standard query 0xdcc8 A b1.nel.goog
    3 0.009246026    10.10.0.1 → 10.10.79.9   DNS 87 Standard query response 0xdcc8 A b1.nel.goog A 142.250.74.67
    4 0.009246437    10.10.0.1 → 10.10.79.9   DNS 131 Standard query response 0xe5b3 HTTPS b1.nel.goog SOA ns1.google.c
om
    5 7.867199479   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0xc900 HTTPS telemetry.canva.com
    6 7.867295491   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0xf408 A telemetry.canva.com
    7 7.884022640    10.10.0.1 → 10.10.79.9   DNS 163 Standard query response 0xf408 A telemetry.canva.com CNAME teleme
try.canva.com.cdn.cloudflare.net A 104.16.103.112 A 104.16.102.112
    8 19.866689309   10.10.79.9 → 10.10.0.1    DNS 75 Standard query 0x38f5 HTTPS ssl.gstatic.com
    9 19.866754643   10.10.79.9 → 10.10.0.1    DNS 75 Standard query 0x700d A ssl.gstatic.com
   10 19.907438569    10.10.0.1 → 10.10.79.9   DNS 132 Standard query response 0x38f5 HTTPS ssl.gstatic.com SOA ns1.goo
gle.com
   11 19.907439030    10.10.0.1 → 10.10.79.9   DNS 91 Standard query response 0x700d A ssl.gstatic.com A 142.251.209.67
   12 38.243481694   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0x920d HTTPS clients4.google.com
   13 38.243550204   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0xe3f5 A clients4.google.com
   14 38.257054348    10.10.0.1 → 10.10.79.9   DNS 119 Standard query response 0xe3f5 A clients4.google.com CNAME clien
ts.l.google.com A 142.251.39.174
   15 38.257054639    10.10.0.1 → 10.10.79.9   DNS 153 Standard query response 0x920d HTTPS clients4.google.com CNAME c
lients.l.google.com SOA ns1.google.com
   16 58.498445791   10.10.79.9 → 10.10.0.1    DNS 74 Standard query 0x4bc1 HTTPS www.google.com
   17 58.498511466   10.10.79.9 → 10.10.0.1    DNS 74 Standard query 0xd298 A www.google.com
   18 58.503803522    10.10.0.1 → 10.10.79.9   DNS 202 Standard query response 0xd298 A www.google.com A 142.251.154.11
9 A 142.251.150.119 A 142.251.152.119 A 142.251.153.119 A 142.251.155.119 A 142.251.156.119 A 142.251.157.119 A 142.251
.151.119
   19 58.510693855   10.10.79.9 → 10.10.0.1    DNS 69 Standard query 0x13d6 HTTPS claude.ai
   20 58.510812871   10.10.79.9 → 10.10.0.1    DNS 69 Standard query 0x07c1 A claude.ai
   21 58.515459153    10.10.0.1 → 10.10.79.9   DNS 85 Standard query response 0x07c1 A claude.ai A 160.79.104.10
   22 62.865730691   10.10.79.9 → 10.10.0.1    DNS 92 Standard query 0x1659 HTTPS browser-intake-us5-datadoghq.com
   23 62.865812235   10.10.79.9 → 10.10.0.1    DNS 92 Standard query 0x1398 A browser-intake-us5-datadoghq.com
   24 62.873100002    10.10.0.1 → 10.10.79.9   DNS 182 Standard query response 0x1659 HTTPS browser-intake-us5-datadogh
q.com SOA ns-cloud-b1.googledomains.com
```

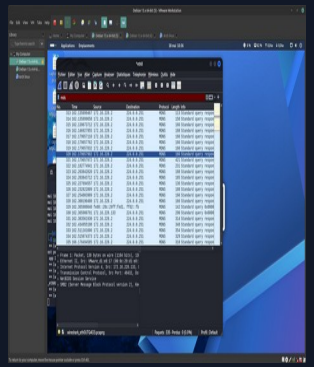
tshark -f "port 53"

mDNS — Port 5353 UDP Multicast

› Sans serveur central — noms .local (Bonjour/Avahi)

› Adresse multicast 224.0.0.251 : 5353

› Découverte imprimantes, partages, services



mDNS multicast

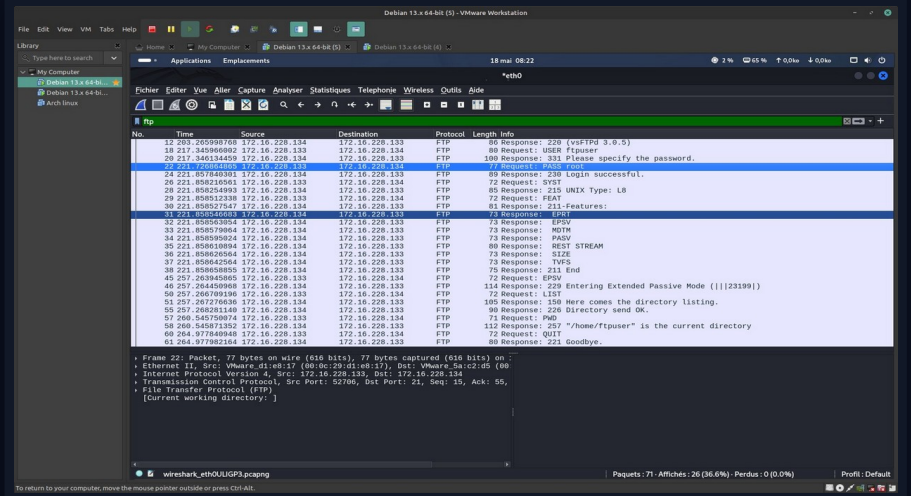
```
sixxla@spamurai:~$ tshark -i wlp2s0 -Y "dns"
tshark -i wlp2s0 -Y "ftp"
tshark -i wlp2s0 -Y "http"
Capturing on 'wlp2s0'
 6109 28.296717049   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0x3ac9 HTTPS a-api.anthropic.co
m
 6110 28.296818652   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0x62dd A a-api.anthropic.com
 6111 28.311463447    10.10.0.1 → 10.10.79.9   DNS 95 Standard query response 0x62dd A a-api.anthrop
ic.com A 160.79.104.10
12940 46.464984769   10.10.79.9 → 10.10.0.1    DNS 77 Standard query 0x93ba HTTPS sb-ssl.google.com
12941 46.465077184   10.10.79.9 → 10.10.0.1    DNS 77 Standard query 0xe665 A sb-ssl.google.com
12942 46.472005820    10.10.0.1 → 10.10.79.9   DNS 164 Standard query response 0xe665 A sb-ssl.googl
e.com CNAME sb-ssl.l.google.com A 192.178.202.190 A 192.178.202.91 A 192.178.202.136 A 192.178.202.9
3
```

tshark -Y "dns"

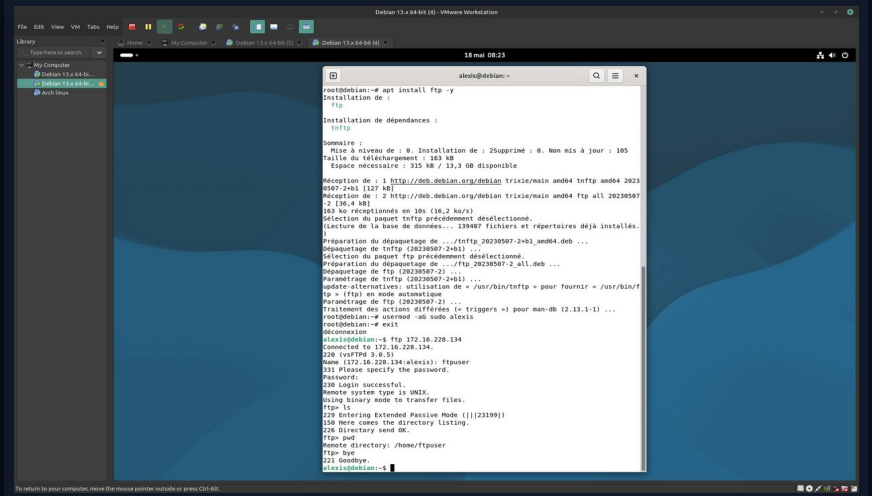
Partie 2 — FTP : Données en clair !

File Transfer Protocol · TCP 21 · vsftpd sur Kali

⚠️ **FTP transmet login ET mot de passe EN CLAIR — interceptables par n'importe qui sur le réseau**



Wireshark — PASS root visible en clair (ligne 22)



Client FTP Debian → connexion réussie LOGIN/PASS

Solution

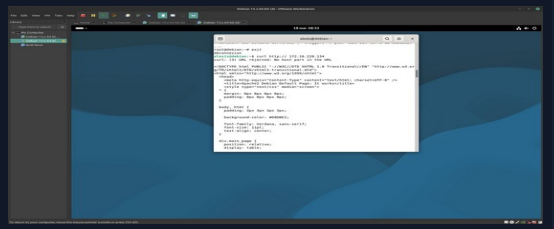
› SFTP — SSH File Transfer Protocol (chiffré par SSH)

› FTPS — FTP over TLS/SSL (chiffré comme HTTPS)

Partie 2 — HTTP • HTTPS • SMB

Protocoles web et partage de fichiers

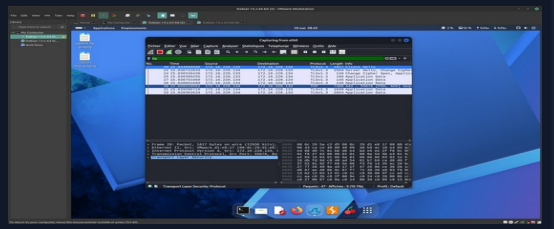
HTTP Port 80 ✗



GET lisible en clair

- GET / HTTP/1.1 en clair
- En-têtes et cookies visibles
- Alternative : HTTPS

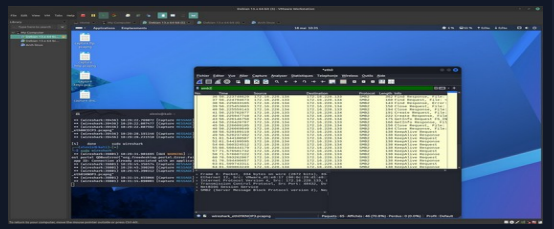
HTTPS / TLS 1.3 ✓



Application Data chiffrée

- Client Hello → Server Hello
- Application Data chiffrée
- Impossible sans clé privée

SMB2 Port 445



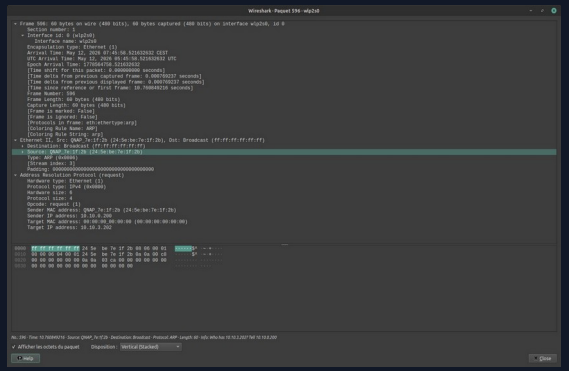
SMB2 — Negotiate/Session/KeepAlive

- Partage fichiers Windows/Linux
- Samba sur Kali → smbclient
- SMBv1 vulnérable WannaCry

Captures annotées — ARP · UDP (DNS) · TCP

Désencapsulation couche par couche · Adresses MAC & IP identifiées

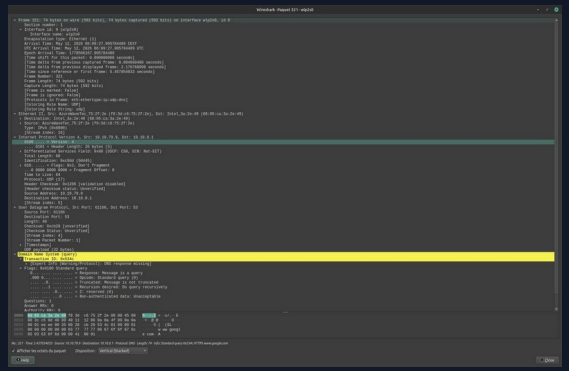
ARP



Requête ARP broadcast

Type	Source	Destination
MAC	24:5e:be:7e:1f:2b	ff:ff:ff:ff:ff:ff
IP	10.10.0.200	10.10.3.202
Proto	ARP 0x0806	Request (opcode=1)

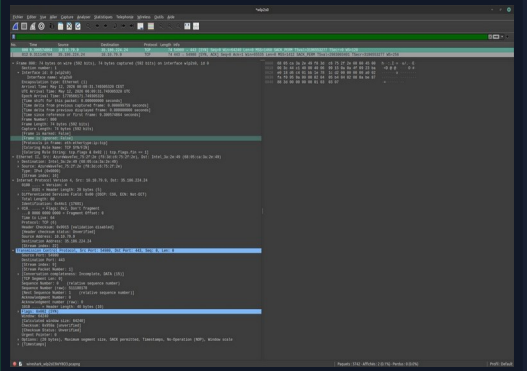
UDP — DNS



Paquet DNS UDP port 53

Type	Source	Destination
MAC	f8:3d:c6:75:2f:2e	68:05:ca:3a:2e:49
IP	10.10.79.9	10.10.0.1
UDP	Port 61166	Port 53 (DNS)

TCP — SYN



SYN / SYN-ACK port 443

Type	Source	Destination
MAC	f8:3d:c6:75:2f:2e	68:05:ca:3a:2e:49
IP	10.10.79.9	35.186.224.24
TCP	Port 54980	Port 443 (HTTPS)

Lecture hexadécimale d'une trame réseau

Correspondance Ethernet + ARP/IPv4 + TCP header · données réelles capturées

HEX DUMP — Trame ARP capturée

Ethernet Header (14 octets)

```
ff ff ff ff ff ff  24 5e be 7e 1f 2b  08 06
```

Dest MAC: ff:ff:ff:ff:ff:ff (broadcast)
Src MAC: 24:5e:be:7e:1f:2b
Type: 0x0806 (ARP)

ARP Header (28 octets)

```
00 01  08 00  06  04  00 01
24 5e be 7e 1f 2b  0a 0a 00 c8
00 00 00 00 00 00  0a 0a 03 ca
```

HW type: 0x0001 (Ethernet)
Proto: 0x0800 (IPv4)
Opcode: 0x0001 (Request)
Sender: 10.10.0.200
Target: 10.10.3.202

IPv4 + TCP Header

IPv4 Header (20 octets)

```
45 00 00 3c  44 c1 40 00
40 06  xx xx  0a 0a 4f 09
23 ba e0 18
```

Version: 4 · IHL: 5 · TTL: 64
Proto: 0x06 (TCP)
Src: 10.10.79.9
Dst: 35.186.224.24

TCP Header (20+ octets)

```
d6 7c  01 bb  1e 82 65 02
00 00 00 00  a0 02  fa f0
xx xx  00 00  [options]
```

Src Port: 54980 (0xd67c)
Dst Port: 443 (0x01bb)
Seq: 511188178
Flags: 0x002 → SYN
Window: 64240

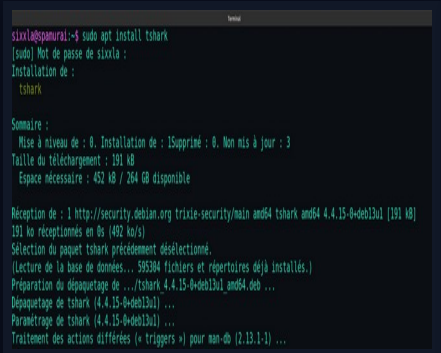
Outil : Wireshark Hex Packet Decoder · chaque octet correspond à un champ précis du protocole

Partie 3 — tshark : options et captures

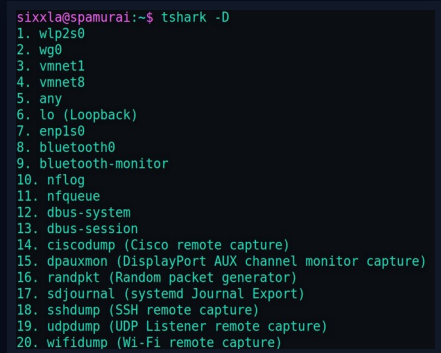
Syntaxe · Filtres BPF et display · Exemples réels

Options principales

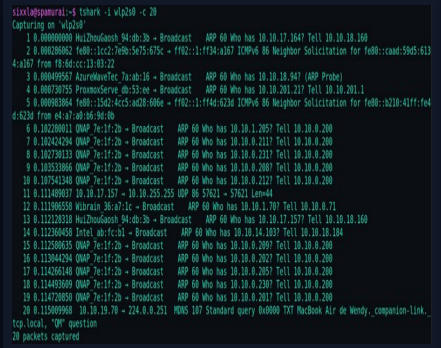
`-i eth0`	Interface réseau à écouter
`-f "port 53"`	Filtre de CAPTURE (BPF) — avant
`-Y "dns"`	Filtre d'AFFICHAGE — après capture
`-c 20`	Limiter à N paquets capturés
`-w dns.pcapng`	Sauvegarder dans un fichier pcapng
`-D`	Lister toutes les interfaces dispo
`-r file.pcapng`	Lire et analyser un fichier existant
`-T fields`	Afficher champs spécifiques (script)



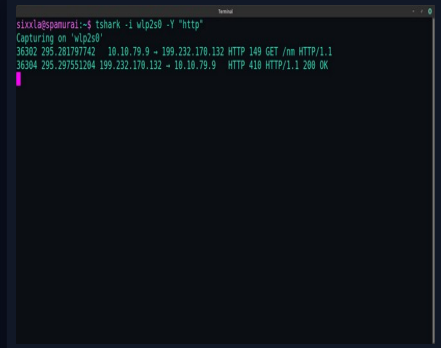
sudo apt install tshark



tshark -D



tshark -c 20



-Y "http"

```
tshark -i wlp2s0 -f "port 53" -Y "dns" -c 50 -w
capture_dns.pcapng
```

Partie 3 — tshark en action

Installation · tshark -D · tshark -c 20

sudo apt install tshark

```
Terminal
sixxla@spamurai:~$ sudo apt install tshark
[sudo] Mot de passe de sixxla :
Installation de :
  tshark

Sommaire :
  Mise à niveau de : 0. Installation de : 1Supprimé : 0. Non mis à jour : 3
Taille du téléchargement : 191 kB
  Espace nécessaire : 452 kB / 264 GB disponible

Réception de : 1 http://security.debian.org trixie-security/main amd64 tshark amd64 4.4.15-0+deb13u1 [191 kB]
191 ko réceptionnés en 0s (492 ko/s)
Sélection du paquet tshark précédemment désélectionné.
(Lecture de la base de données... 595304 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../tshark_4.4.15-0+deb13u1_amd64.deb ...
Dépaquetage de tshark (4.4.15-0+deb13u1) ...
Paramétrage de tshark (4.4.15-0+deb13u1) ...
Traitement des actions différées (« triggers ») pour man-db (2.13.1-1) ...
```

tshark -D → Lister les interfaces disponibles

```
sixxla@spamurai:~$ tshark -D
1. wlp2s0
2. wg0
3. vmnet1
4. vmnet8
5. any
6. lo (Loopback)
7. enp1s0
8. bluetooth0
9. bluetooth-monitor
10. nflog
11. nfqueue
12. dbus-system
13. dbus-session
14. ciscodump (Cisco remote capture)
15. dpauxmon (DisplayPort AUX channel monitor capture)
16. randpkt (Random packet generator)
17. sdjournal (systemd Journal Export)
18. sshdump (SSH remote capture)
19. udpdump (UDP Listener remote capture)
20. wifidump (Wi-Fi remote capture)
```

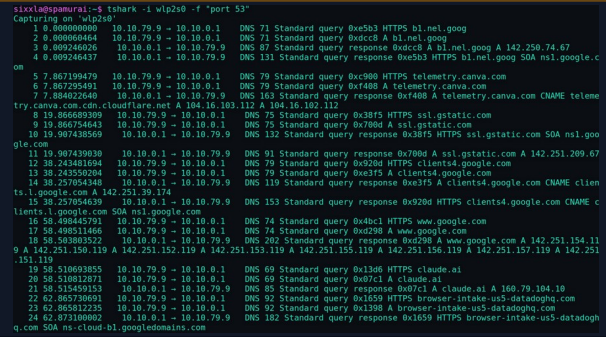
tshark -i wlp2s0 -c 20 → Capturer 20 paquets (ARP, MDNS, UDP visibles)

```
sixxla@spamurai:~$ tshark -i wlp2s0 -c 20
Capturing on 'wlp2s0'
    1 0.000000000 HuiZhouGaosh_94:db:3b → Broadcast    ARP 60 Who has 10.10.17.164? Tell 10.10.18.160
    2 0.000286062 fe80::1cc2:7e9b:5e75:675c → ff02::1:ff34:a167 ICMPv6 86 Neighbor Solicitation for fe80::caad:59d5:6134:a167 from f0:6d:cc:13:03:22
    3 0.000499567 AzureWaveTec_7a:ab:16 → Broadcast    ARP 60 Who has 10.10.18.94? (ARP Probe)
    4 0.000730755 ProxmoxServe_db:53:ee → Broadcast    ARP 60 Who has 10.10.201.21? Tell 10.10.201.1
    5 0.000983864 fe80::15d2:4cc5:ad28:606e → ff02::1:ff4d:623d ICMPv6 86 Neighbor Solicitation for fe80::b210:41ff:fe4d:623d from e4:a7:a0:b6:9d:0b
    6 0.102280011 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.1.205? Tell 10.10.0.200
    7 0.102424294 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.211? Tell 10.10.0.200
    8 0.102730133 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.231? Tell 10.10.0.200
    9 0.103533866 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.208? Tell 10.10.0.200
   10 0.107541348 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.212? Tell 10.10.0.200
   11 0.111489037 10.10.17.157 → 10.10.255.255 UDP 86 57621 → 57621 Len=44
   12 0.111906558 Wibrain_36:a7:1c → Broadcast    ARP 60 Who has 10.10.1.70? Tell 10.10.0.71
   13 0.112128318 HuiZhouGaosh_94:db:3b → Broadcast    ARP 60 Who has 10.10.17.157? Tell 10.10.18.160
   14 0.112360458 Intel_ab:fc:b1 → Broadcast    ARP 60 Who has 10.10.14.103? Tell 10.10.18.184
   15 0.112580635 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.209? Tell 10.10.0.200
   16 0.113044294 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.202? Tell 10.10.0.200
   17 0.114266148 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.205? Tell 10.10.0.200
   18 0.114493609 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.230? Tell 10.10.0.200
   19 0.114720850 QNAP_7e:1f:2b → Broadcast    ARP 60 Who has 10.10.0.201? Tell 10.10.0.200
   20 0.115009960  10.10.19.70 → 224.0.0.251  MDNS 107 Standard query 0x0000 TXT MacBook Air de Wendy._companion-link._tcp.local, "QM" question
20 packets captured
```

Partie 3 — tshark : filtres en action

-f "port 53" · -Y "dns" · -Y "http"

tshark -i wlp2s0 -f "port 53" → Filtre BPF AVANT la capture — seuls les paquets DNS sont enregistrés

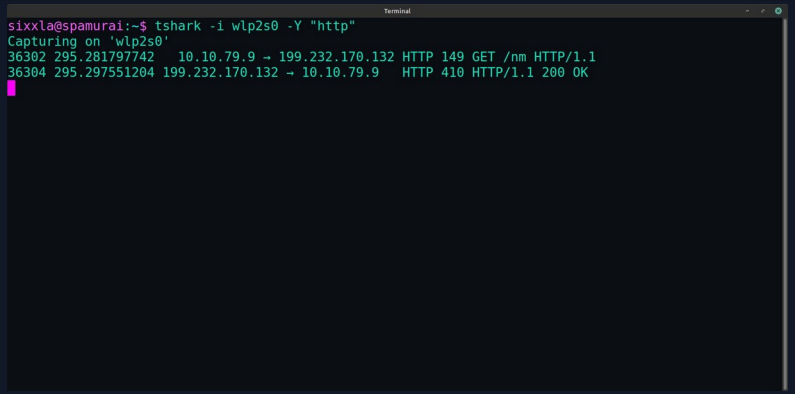


```
sixxla@spamurai:~$ tshark -i wlp2s0 -f "port 53"
Capturing on 'wlp2s0'
    1 0.000000000   10.10.79.9 → 10.10.0.1    DNS 71 Standard query 0xe5b3 HTTPS b1.nel.goog
    2 0.000060464   10.10.79.9 → 10.10.0.1    DNS 71 Standard query 0xdcc8 A b1.nel.goog
    3 0.009246026    10.10.0.1 → 10.10.79.9   DNS 87 Standard query response 0xdcc8 A b1.nel.goog A 142.250.74.67
    4 0.009246437    10.10.0.1 → 10.10.79.9   DNS 131 Standard query response 0xe5b3 HTTPS b1.nel.goog SOA ns1.google.c
om
    5 7.867190479   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0xc900 HTTPS telemetry.canva.com
    6 7.867295491   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0xf408 A telemetry.canva.com
    7 7.884022640    10.10.0.1 → 10.10.79.9   DNS 163 Standard query response 0xf408 A telemetry.canva.com CNAME teleme
try.canva.com.cdn.cloudflare.net A 104.16.103.112 A 104.16.102.112
    8 19.866689309   10.10.79.9 → 10.10.0.1    DNS 75 Standard query 0x38f5 HTTPS ssl.gstatic.com
    9 19.866754643   10.10.79.9 → 10.10.0.1    DNS 75 Standard query 0x700d A ssl.gstatic.com
   10 19.907438569    10.10.0.1 → 10.10.79.9   DNS 132 Standard query response 0x38f5 HTTPS ssl.gstatic.com SOA ns1.goo
gle.com
   11 19.907439030    10.10.0.1 → 10.10.79.9   DNS 91 Standard query response 0x700d A ssl.gstatic.com A 142.251.209.67
   12 38.243481694   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0x920d HTTPS clients4.google.com
   13 38.243550204   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0xe3f5 A clients4.google.com
   14 38.257054348    10.10.0.1 → 10.10.79.9   DNS 119 Standard query response 0xe3f5 A clients4.google.com CNAME clien
ts.l.google.com A 142.251.39.174
   15 38.257054639    10.10.0.1 → 10.10.79.9   DNS 153 Standard query response 0x920d HTTPS clients4.google.com CNAME c
lients.l.google.com SOA ns1.google.com
   16 58.498445791   10.10.79.9 → 10.10.0.1    DNS 74 Standard query 0x4bc1 HTTPS www.google.com
   17 58.498511466   10.10.79.9 → 10.10.0.1    DNS 74 Standard query 0xd298 A www.google.com
   18 58.503803522    10.10.0.1 → 10.10.79.9   DNS 202 Standard query response 0xd298 A www.google.com A 142.251.154.11
9 A 142.251.150.119 A 142.251.152.119 A 142.251.153.119 A 142.251.155.119 A 142.251.156.119 A 142.251.157.119 A 142.251
.151.119
   19 58.510693855   10.10.79.9 → 10.10.0.1    DNS 69 Standard query 0x13d6 HTTPS claude.ai
   20 58.510812871   10.10.79.9 → 10.10.0.1    DNS 69 Standard query 0x07c1 A claude.ai
   21 58.515459153    10.10.0.1 → 10.10.79.9   DNS 85 Standard query response 0x07c1 A claude.ai A 160.79.104.10
   22 62.865730691   10.10.79.9 → 10.10.0.1    DNS 92 Standard query 0x1659 HTTPS browser-intake-us5-datadoghq.com
   23 62.865812235   10.10.79.9 → 10.10.0.1    DNS 92 Standard query 0x1398 A browser-intake-us5-datadoghq.com
   24 62.873100002    10.10.0.1 → 10.10.79.9   DNS 182 Standard query response 0x1659 HTTPS browser-intake-us5-datadogh
q.com SOA ns-cloud-b1.googledomains.com
```

tshark -i wlp2s0 -Y "dns" → Display filter APRÈS

```
sixxla@spamurai:~$ tshark -i wlp2s0 -Y "dns"
tshark -i wlp2s0 -Y "ftp"
tshark -i wlp2s0 -Y "http"
Capturing on 'wlp2s0'
 6109 28.296717049   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0x3ac9 HTTPS a-api.anthropic.co
m
 6110 28.296818652   10.10.79.9 → 10.10.0.1    DNS 79 Standard query 0x62dd A a-api.anthropic.com
 6111 28.311463447    10.10.0.1 → 10.10.79.9   DNS 95 Standard query response 0x62dd A a-api.anthrop
ic.com A 160.79.104.10
12940 46.464984769   10.10.79.9 → 10.10.0.1    DNS 77 Standard query 0x93ba HTTPS sb-ssl.google.com
12941 46.465077184   10.10.79.9 → 10.10.0.1    DNS 77 Standard query 0xe665 A sb-ssl.google.com
12942 46.472005820    10.10.0.1 → 10.10.79.9   DNS 164 Standard query response 0xe665 A sb-ssl.googl
e.com CNAME sb-ssl.l.google.com A 192.178.202.190 A 192.178.202.91 A 192.178.202.136 A 192.178.202.9
3
```

tshark -i wlp2s0 -Y "http" → Requêtes HTTP en clair



```
Terminal
sixxla@spamurai:~$ tshark -i wlp2s0 -Y "http"
Capturing on 'wlp2s0'
36302 295.281797742   10.10.79.9 → 199.232.170.132 HTTP 149 GET /nm HTTP/1.1
36304 295.297551204 199.232.170.132 → 10.10.79.9   HTTP 410 HTTP/1.1 200 OK
```

Sécurité — Protocoles chiffrés vs non chiffrés

FTP/HTTP en clair · HTTPS/TLS chiffré · Alternatives sécurisées

Protocole	Chiffrement	Login	Risque	Alternative
FTP	AUCUN	OUI ✗	🔴 Élevé	SFTP / FTPS
HTTP	AUCUN	OUI ✗	🔴 Élevé	HTTPS
DNS	AUCUN	—	🟡 Moyen	DoH / DoT
DHCP	AUCUN	—	🟡 Moyen	—
SMB2	Partiel	—	🟡 Moyen	SMB3+Kerb
HTTPS	TLS 1.3	NON ✓	🟢 Faible	✓ OK
FTPS	TLS	NON ✓	🟢 Faible	✓ OK
SFTP	SSH	NON ✓	🟢 Faible	✓ OK

FTP sans TLS

› Login et mdp visibles en clair

› Interceptable par n'importe qui

› Dangereux sur tout réseau partagé

SSL / TLS (HTTPS)

› Handshake TLS (Client/Server Hello)

› Application Data chiffrée

› Clé privée indispensable

Conclusion

Bilan du projet · Compétences acquises

Wireshark & tshark

Capture, filtrage et analyse de trafic réseau en temps réel (GUI + CLI)

Modèle OSI

Désencapsulation couche par couche — Trame L2, Paquet L3, Segment L4, Application L7

Protocoles analysés

ARP, UDP, TCP, DHCP, DNS, mDNS, FTP, HTTP, HTTPS, TLS v1.3, SMB2 — tous documentés

Sécurité réseau

FTP/HTTP : login/mdp en clair · HTTPS/TLS : chiffrement end-to-end · Importance des protocoles sécurisés

Scripts bash

tshark intégré dans des pipelines pour l'analyse et la surveillance automatisée du réseau

Compétences visées · Administrer l'infrastructure réseau · Analyser la sécurité · Détecter les incidents · Élaborer la politique de sécurité